Machine Learning

Data Science

Course Schedule

1. Exploratory Data Analysis
2. Clustering
3. Probabilistic Models
4. Statistical Inference
5. Machine Learning
6. Feature Engineering
7. Model Evaluation
8. Deep Learning
9. Time Series
10. Causality

Machine Learning

- linear regression
- training
- logistic regression
- Generalized Linear Models
- quantile regression
- Generalized Additive Models
- k-Nearest Neighbors
- decision trees and ensemble methods

What Is Machine Learning?

learning from experience/data: exploiting statistical dependencies with the aim of **generalization** to new data

training: **ML algorithm + data = explicit algorithm** (to be used later)

→ reduction of complexity and much better generalizability compared to handcrafted algorithms

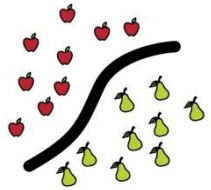
analogy: Humans do not hit the ground running (storage capacity of DNA limited) but have learning capabilities.

MACHINE LEARNING

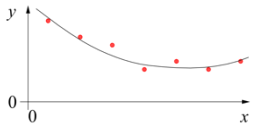
training target available (labeled or past data)

SUPERVISED

CLASSIFICATION



REGRESSION



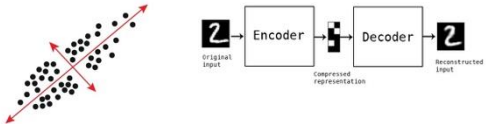
data not labeled in any way

UNSUPERVISED

CLUSTERING

SELF-SUPERVISED

DIMENSIONALITY REDUCTION



ASSOCIATION

no supervision, but goal-based interaction with environment

REINFORCEMENT LEARNING

LEARN STATE OR ACTION VALUES

LEARN POLICY DIRECTLY

learning by teacher (high-dimensional curve fitting)

learning by observation (pattern recognition)

learning by trial-and-error (sequential decision making)

Supervised Learning

Target Quantity

- **known in training**: labeled samples or observations from past
- to be **predicted** for unknown cases (e.g., future values)

Features

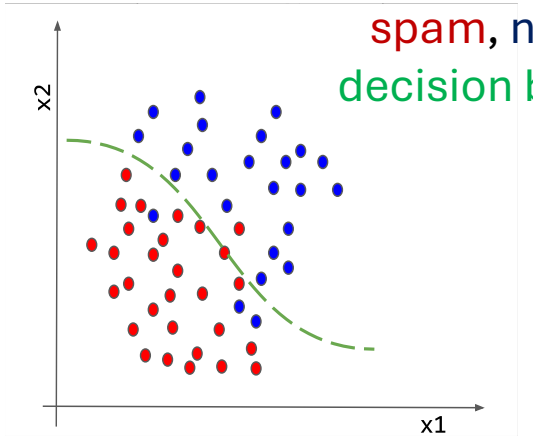
(prepared) input information that is
- correlated to target quantity
- known at prediction time

Example: Spam Filtering

classify emails as spam or no spam

*use accordingly **labeled emails as training set***

use information like occurrence of specific words or email length as **features**



Supervised Learning in Mathematical Terms

map input to output: $y = f(\boldsymbol{x})$

random variables Y and $\boldsymbol{X} = (X_1, X_2, \cdots, X_p)$ ← usually high-dimensional

training (e.g., curve fitting / parameter estimation):

fit data set of $(y_i, \boldsymbol{x}_i)$ pairs → minimize deviations between y and $f(\boldsymbol{x})$

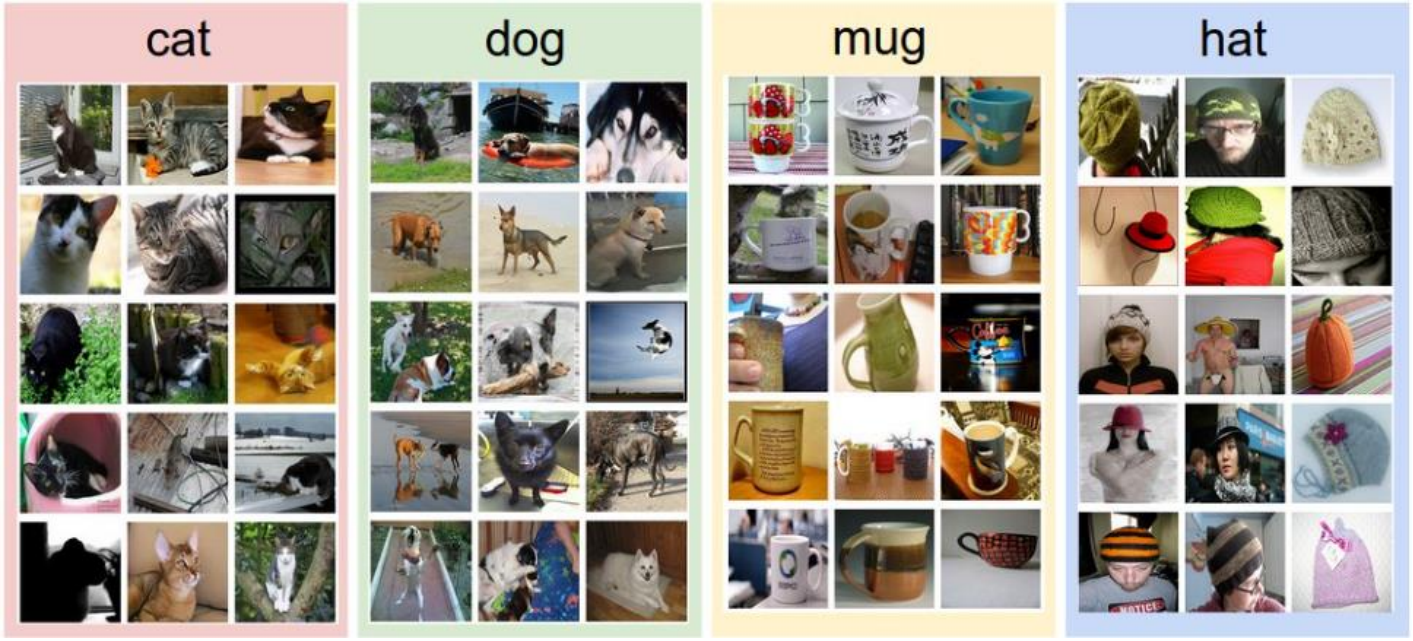
discriminative models: predict conditional density function $p(y|\boldsymbol{x})$

as opposed to generative models: predict $p(y, \boldsymbol{x})$ (or just $p(\boldsymbol{x})$) → $\boldsymbol{x}$ not given, more difficult

Example: Image Classification

training data set:

labels:



test data with learned classifier:

$f($  $) =$ “Cat”

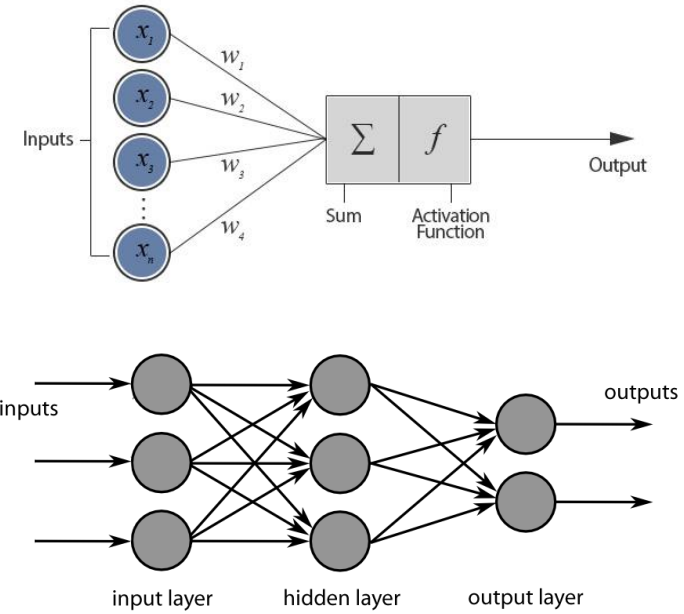
$f($  $) =$ “Dog”

Algorithmic Families of Supervised Learning

parametric models

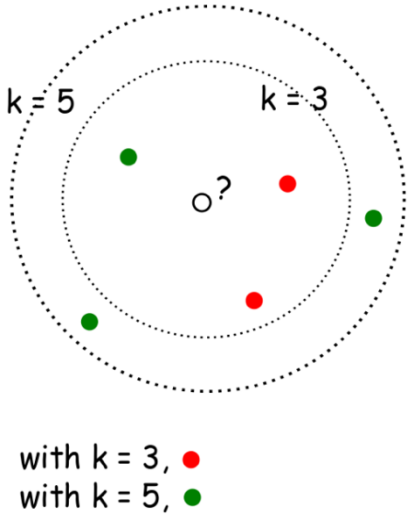
linear models

neural networks



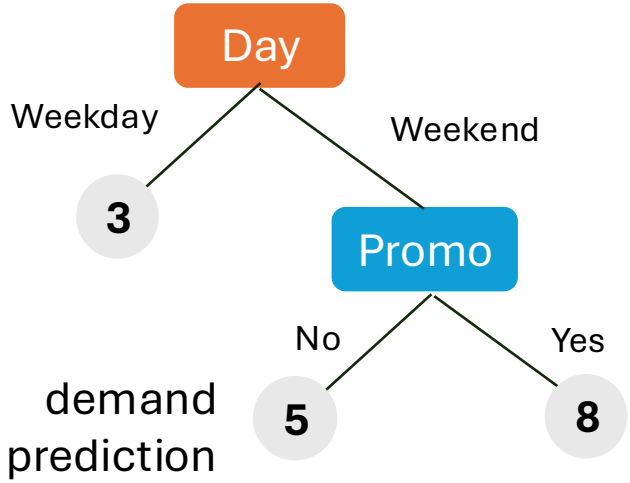
non-parametric models: similarity-based learning (analogies)

nearest-neighbor-style/local methods



kernel machines (e.g., support-vector machines)

decision trees: rule learning



often used in ensemble methods

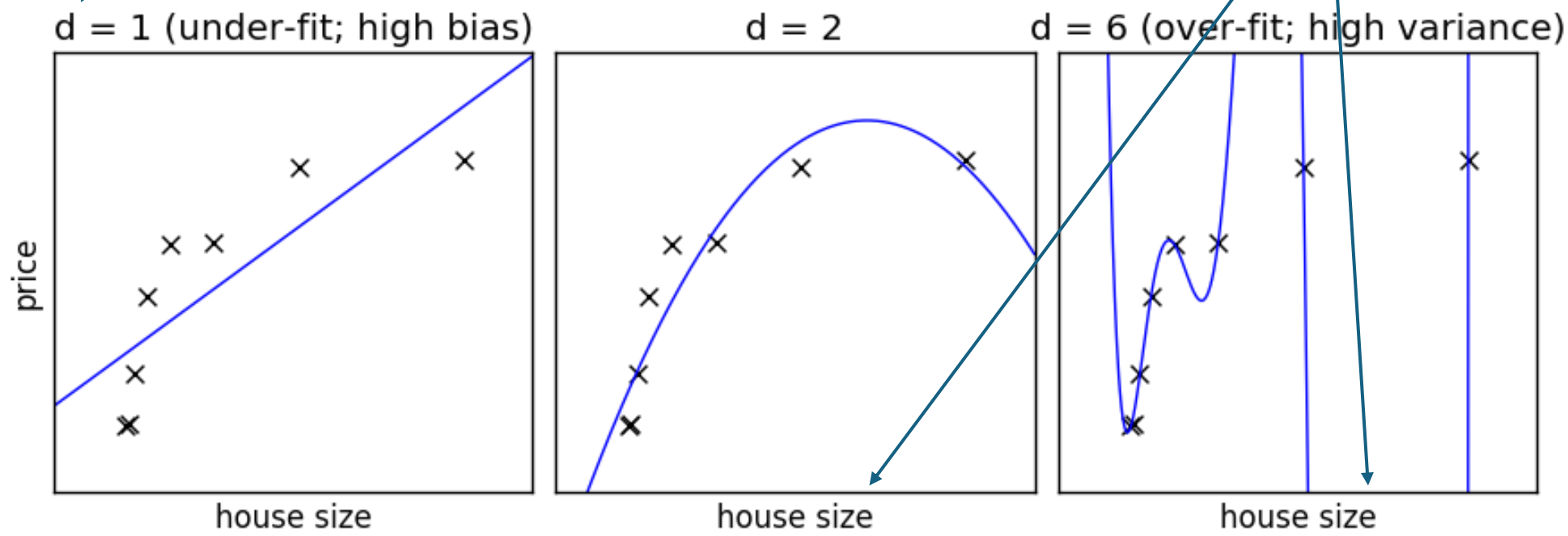
- bagging: random forests
- boosting: gradient boosting

Under- and Over-Fitting

example: polynomial fit

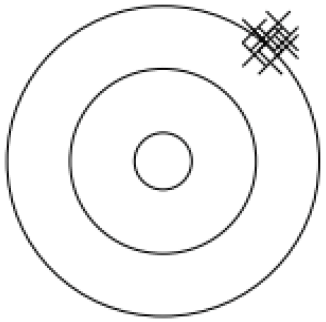
need to predict new samples (not in training data set): interpolation

degree of fitted polynomial

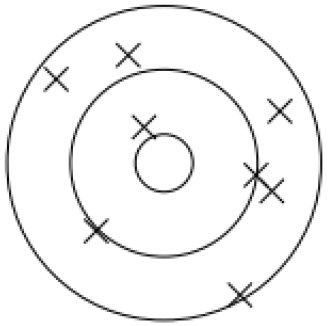


Bias, Variance, Irreducible Error

think of fitting ML algorithms as repeatable processes with different data sets



bias
due to too simplistic model
(same for all training data sets)
“underfitting”



variance
due to sensitivity to specifics (noise)
of different training data sets
“overfitting”

irreducible error (aka Bayes error):

inherent randomness (target generated from random variable)

→limiting accuracy of ideal model

Bias-Variance Tradeoff

models of higher complexity have lower bias but higher variance

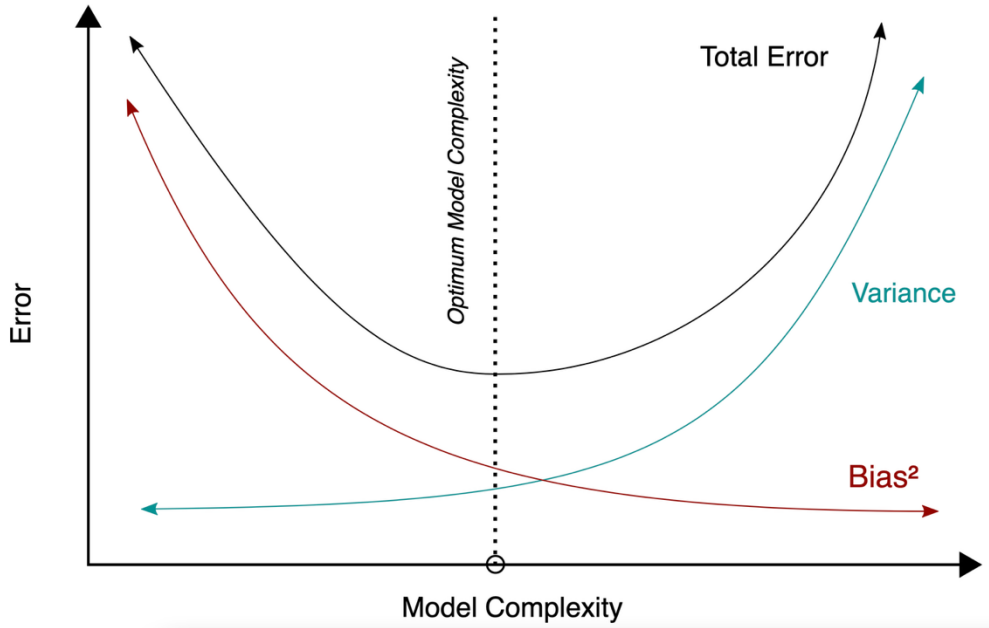
(given the same number of training examples)

generalization error follows U-shaped curve: overfitting once model complexity (number of parameters) passes certain threshold

overfitting: variance term dominating test error

→ increasing model complexity increases test error

but beware: training error keeps decreasing with more complex model …

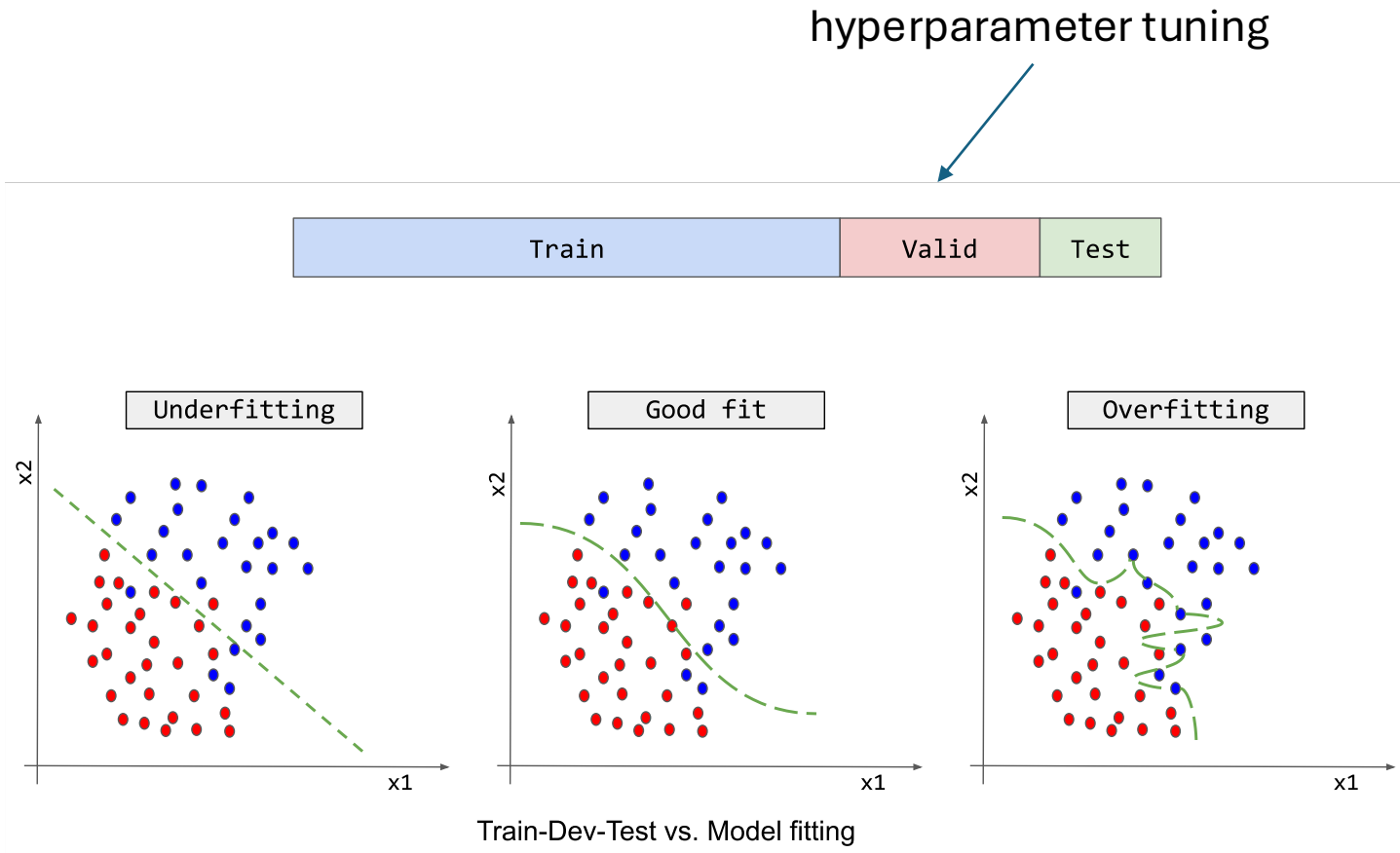


Hyperparameters

model complexity often controlled via hyperparameters (parameters not fitted but set in advance)

examples:

- degree of fitted polynomial d
- number of considered nearest neighbors k
- maximum depth of decision tree
- number of hidden nodes in neural network layer



Train-Dev-Test vs. Model fitting

Generalization

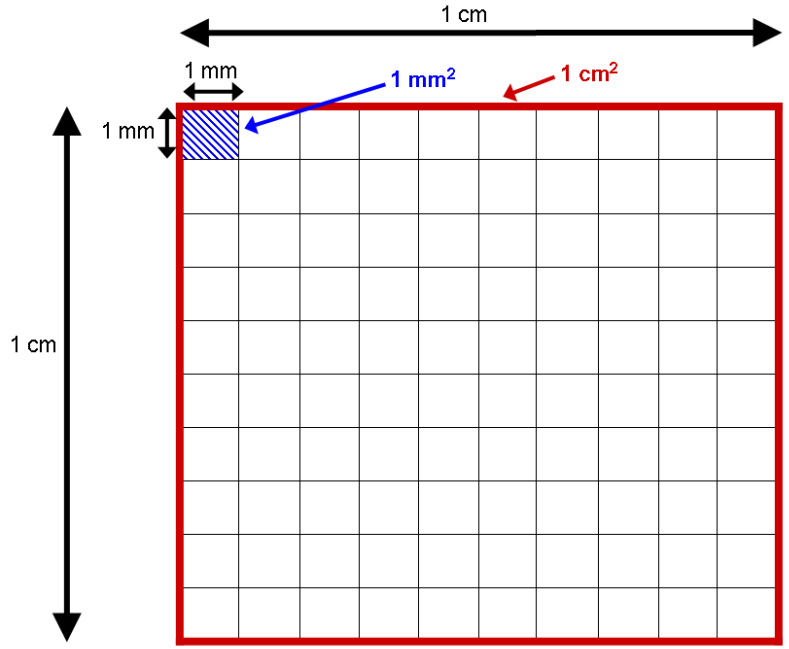
core of ML: **empirical risk minimization** (training error) as proxy for minimizing unknown population risk (test error)

generalization gap: difference between test and training error

curse of dimensionality: typically, many features (dimensions)

→ lots of data needed to densely sample volume

Curse of Dimensionality



data sparsity:

dimensions (i.e., features) increase

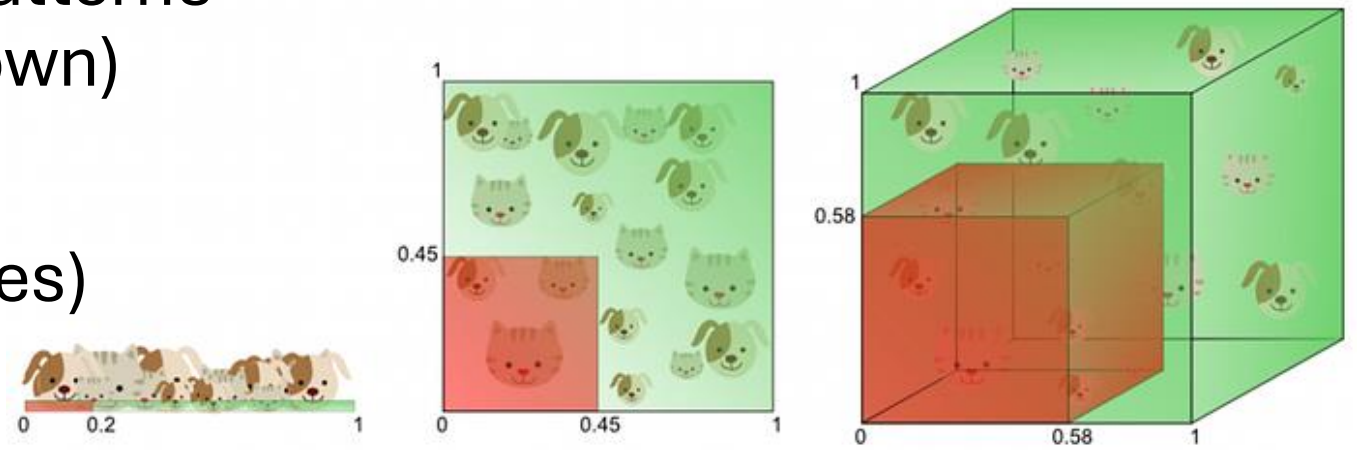
→ volume of space increases rapidly

→ data points become isolated

→ difficult to find meaningful patterns (concept of proximity breaks down)

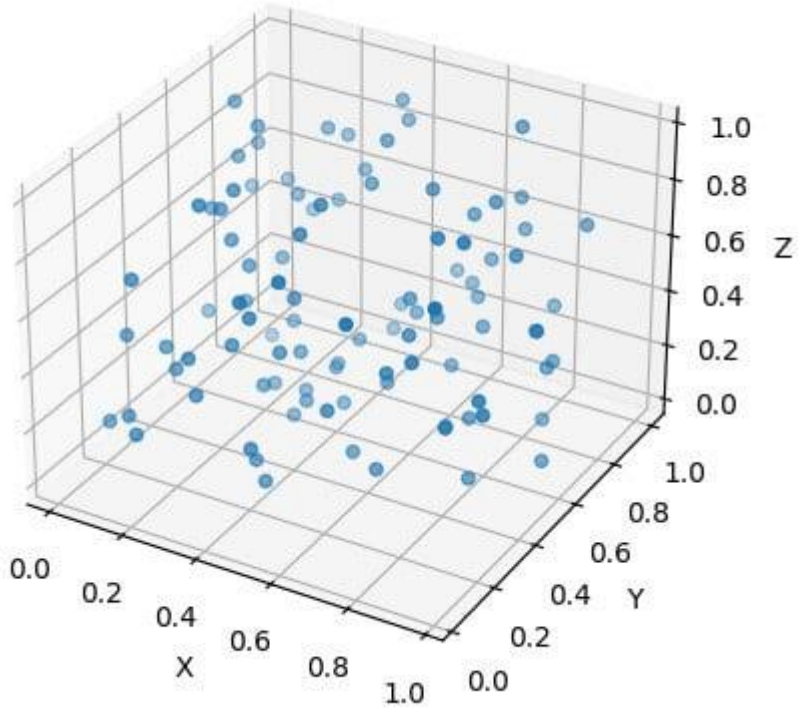
→ overfitting

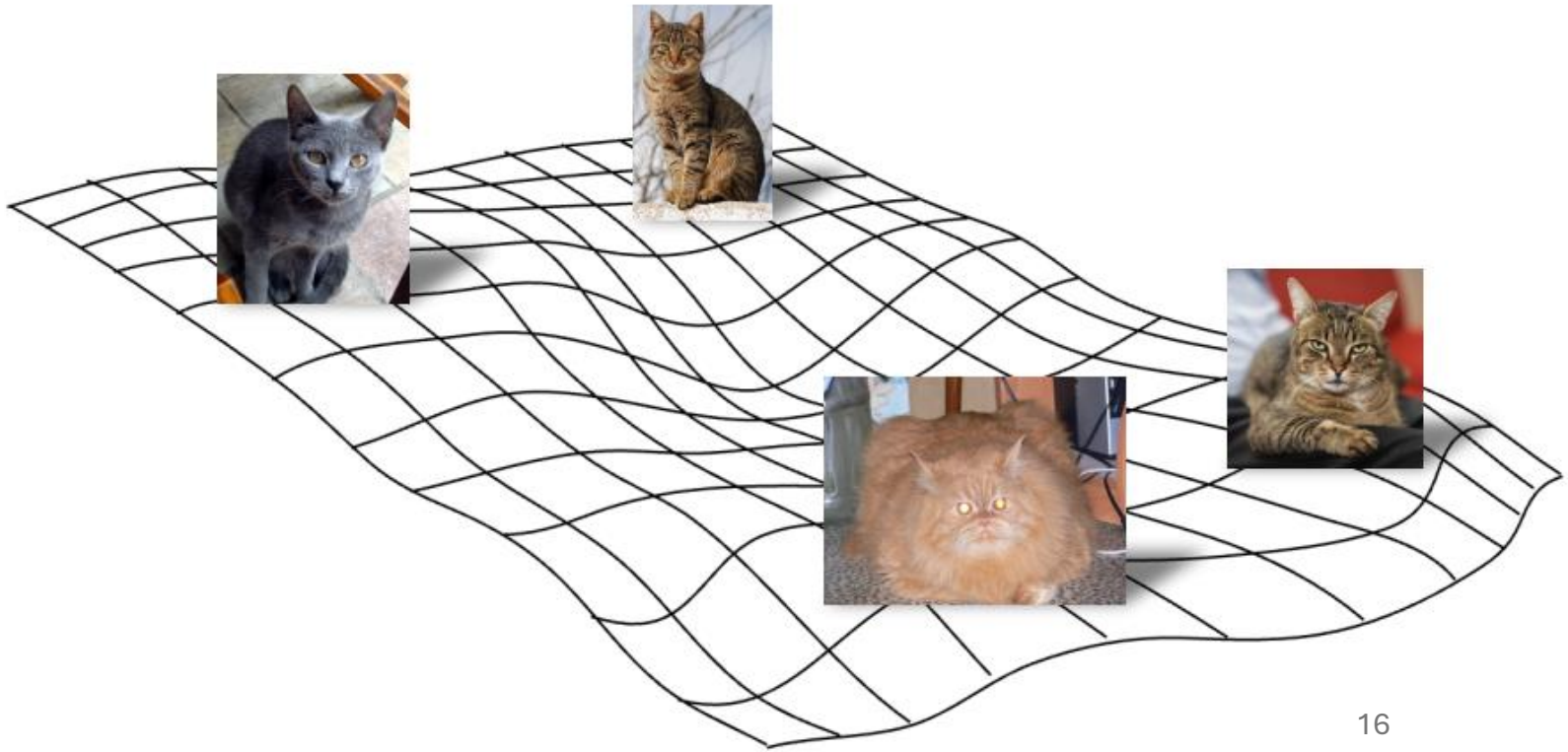
→ exponential data (i.e., samples) requirements



Manifold Hypothesis

luckily, reality is friendly: most high-dimensional data sets reside on lower-dimensional manifolds → enabling effectiveness of ML





Linear Regression

fit:

$$y_i = b + \sum_{j=1}^{p} w_j \, x_{ij} + \varepsilon_i \qquad f(\boldsymbol{x}_i)$$

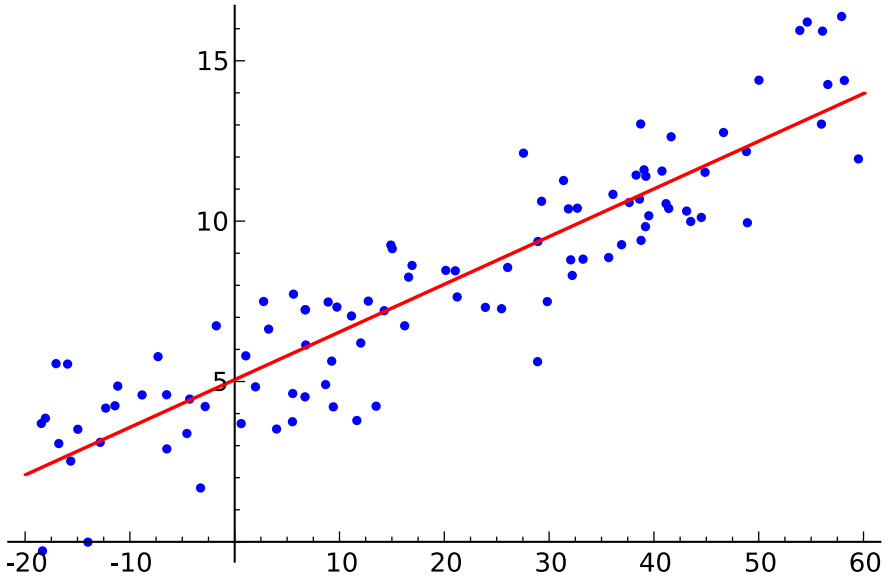
predict:

$$\hat{y}_i = E[Y|\boldsymbol{X} = \boldsymbol{x}_i] = f(\boldsymbol{x}_i) = b + \sum_{j=1}^{p} w_j \, x_{ij}$$

Gaussian

$$p(y|\boldsymbol{x}_i) = \mathcal{N}(y; \hat{y}_i, \sigma^2)$$

Gaussian mean variance



parameter estimation:
e.g., least squares

parameters to be estimated: $b, \boldsymbol{w}$

$\rightarrow \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - f(\boldsymbol{x}_i)\right)^2$

General Recipe of Statistical Learning

ML algorithm by combining:

- **model** (e.g., linear function & Gaussian distribution)
- **objective function** (e.g., squared residuals)
- **optimization algorithm** (e.g., gradient descent)
- **regularization** (e.g., L2, dropout)

Linear Model

data: vector with p different feature values for this example i

$$f(\boldsymbol{x}_i) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

vector with slope parameters (one for each of the p features, but identical for all examples), aka weights

single intercept parameter, aka bias

Loss Function

loss function L: expressing deviation between prediction $\hat{y}$ and target y

$$L(y_i, \hat{y}_i) = L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{w})\big)$$

with $\boldsymbol{w}$ corresponding to parameters of model $f(\boldsymbol{x}_i)$

(for the sake of clarity, include bias b in $\boldsymbol{w}$)

prime example: squared residuals for regression problems

$$L(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

Cost Function

averaging losses over empirical training data set:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{w})\big)$$

cost function to be minimized according to model parameters $\boldsymbol{w}$

→ objective function

Cost Minimization

minimize training costs $\mathcal{L}(\boldsymbol{w})$ according to model parameters $\boldsymbol{w}$:

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = 0$$

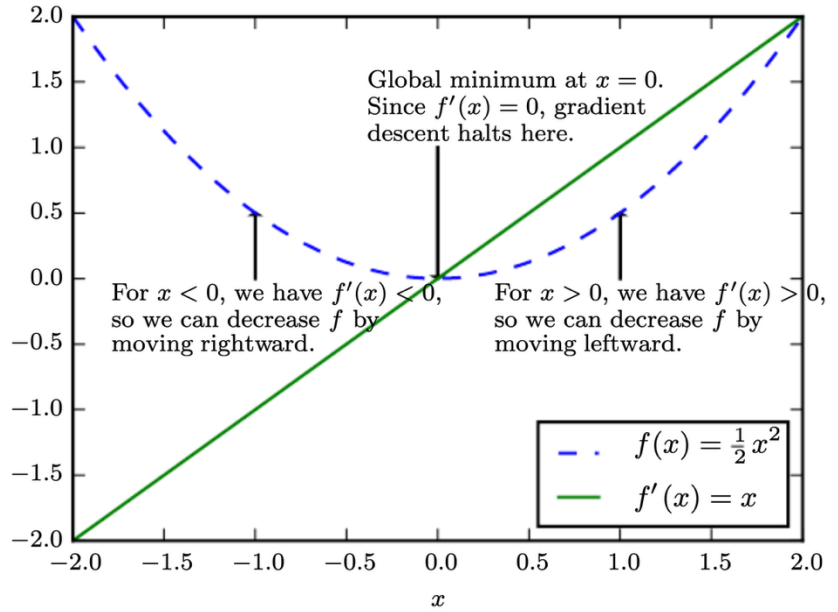
for mean squared error (aka least squares method):

$$\nabla_{\boldsymbol{w}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{w}) \right)^2 = 0$$

Gradient Descent

typically no closed-form solution to minimization of cost function: $\nabla_{\boldsymbol{w}}\mathcal{L} = 0$

→ need for numerical methods



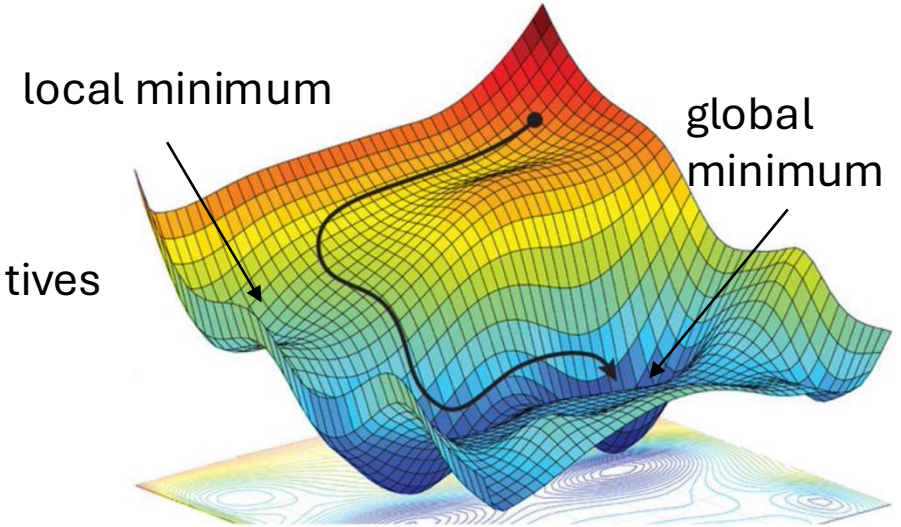
source

popular choice: gradient descent

iteratively moving in direction of steepest descent (negative gradient): $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\nabla_{w}\mathcal{L}$

step size (learning rate)

vector containing all partial derivatives



L^2 Regularization (Ridge Regression)

add **penalty term** on parameter L^2 norm to cost function

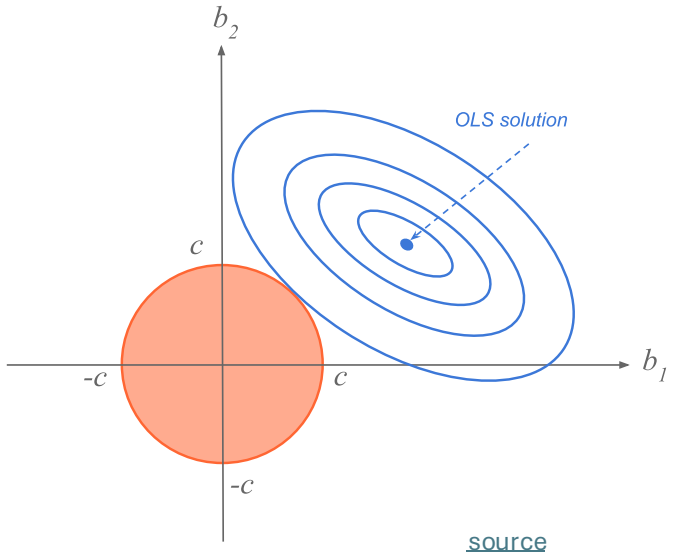
$$\tilde{\mathcal{L}}(\boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \lambda \cdot \boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}$$

hyperparameter controlling strength of regularization

aka **weight decay** in neural networks

corresponds to imposing constraint on parameters to lie in L^2 region (size controlled by λ)

goal: reduce overfitting



source

Classification: Logistic Regression

binary classification: $y = 1$ or $y = 0$

→ predict probabilities p_i for $y = 1$

- logit (log-odds) as link function
- Y following Bernoulli distribution

$$\text{logit}(E[Y|\boldsymbol{X} = \boldsymbol{x}_i]) = \ln\left(\frac{p_i}{1 - p_i}\right) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

Multi-Classification: Softmax

for classification in K categories:

- use “score” function with K outputs

$$f_k(\boldsymbol{x}_i) = \boldsymbol{w}_k \cdot \boldsymbol{x}_i + b_k$$

- and convert into probabilities by means of softmax function

$$\frac{\exp\left(f_k(\boldsymbol{x}_i)\right)}{\sum_{l=1}^{K} \exp\left(f_l(\boldsymbol{x}_i)\right)}$$

Generalized Linear Models (GLMs)

$$g(E[Y|\boldsymbol{X} = \boldsymbol{x}_i]) = \hat{\alpha} + \sum_{j=1}^{p} \hat{\beta}_j \, x_{ij}$$

link function g:

- linking range of Y to linear predictor
- canonical forms for different Y distributions
  - log for Poisson
  - identity for Gaussian
    → linear regression)

Y following probability distribution from exponential family (e.g., Poisson or Gaussian)

Multiplicative Model

- count data: $Y \in [0, \infty)$
- Y follows Poisson (or negative binomial / Poisson-gamma) distribution

log-linear model (Gaussian errors in fit, Poisson with mean $\hat{y}_i$ predicted):

single parameter

link function

$$\log(\underbrace{E[Y|\boldsymbol{X} = \boldsymbol{x}_i]}_{\hat{y}_i}) = \hat{\alpha} + \sum_{j=1}^{p} \hat{\beta}_j \, x_{ij}$$

- further advantage: usually multiplicative effects for count data, i.e., variation proportional to level (small effects for small counts, large effects for large counts)

Generalized Additive Models (GAMs)

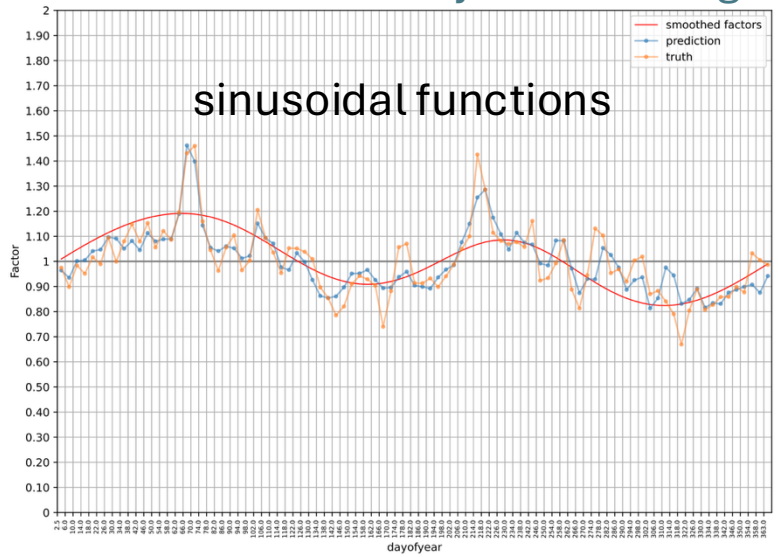
plots from
Cyclic Boosting

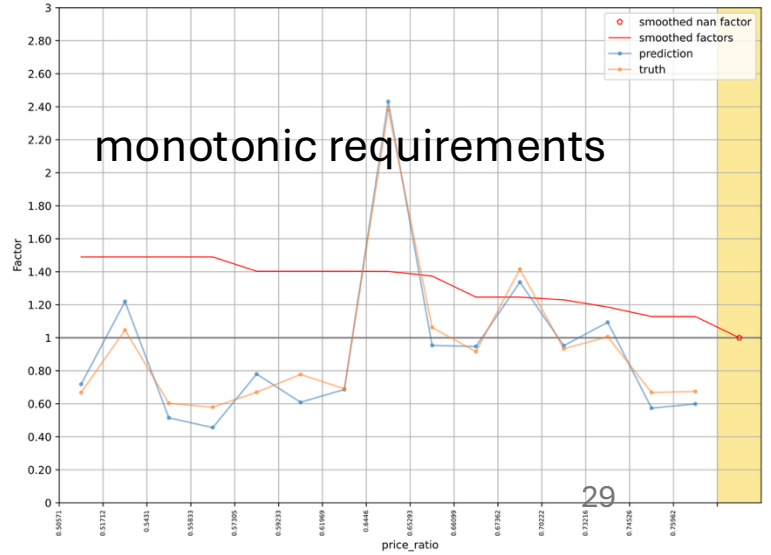
blending of Generalized Linear Models and additive models

$$g(E[Y|\boldsymbol{X} = \boldsymbol{x}_i]) = \hat{\alpha} + \sum_{j=1}^{p} \hat{h}_j(x_{ij})$$

smooth functions

- potentially non-parametric form
- describe non-linear effects
- estimated, e.g., via backfitting algorithm
- extension: add interaction terms between different features (e.g., different holiday effects for different products)





Quantile Regression

estimate quantile τ of distribution instead of conditional mean by minimizing pinball loss instead of squared error loss (choice of loss function defines point estimate):

$$(1-\tau)\sum_{y_i<\hat{q}_i}(\hat{q}_i - y_i) + \tau\sum_{y_i\geq\hat{q}_i}(y_i - \hat{q}_i)$$

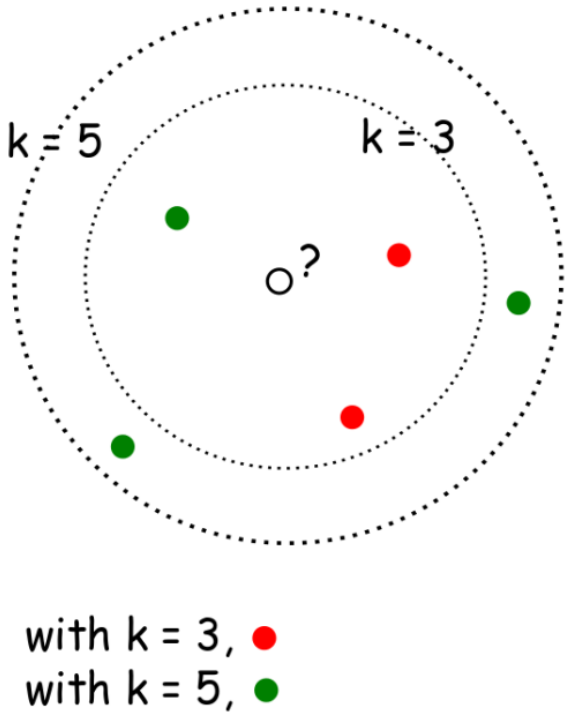
possible with various ML methods, including neural networks and tree-based methods (like random forests or gradient boosting)

different options for prediction of full probability distribution:

- approximate from a few predicted quantiles by fitting (assumed distribution or spline) or quantile-parameterized distributions
- PDF assumption for model and estimation of its parameters

k-Nearest Neighbors (kNN)

- local method, instance-based learning
- non-parametric
- distance defined by metric on $\boldsymbol{x}$ (e.g., Euclidean)



regression:

$$f(\boldsymbol{x}_0) = \frac{1}{k}\sum_{j=1}^{k} y_j$$

with j running over k nearest neighbors of $\boldsymbol{x}_0$

low k: low bias but high variance
high k: low variance but high bias

Image Classification with kNN

choose an image distance, e.g., L1 distance:

$$d(\boldsymbol{I}_1, \boldsymbol{I}_2) = \sum_p |I_1(p) - I_2(p)|$$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

-

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

=

pixel-wise absolute value differences

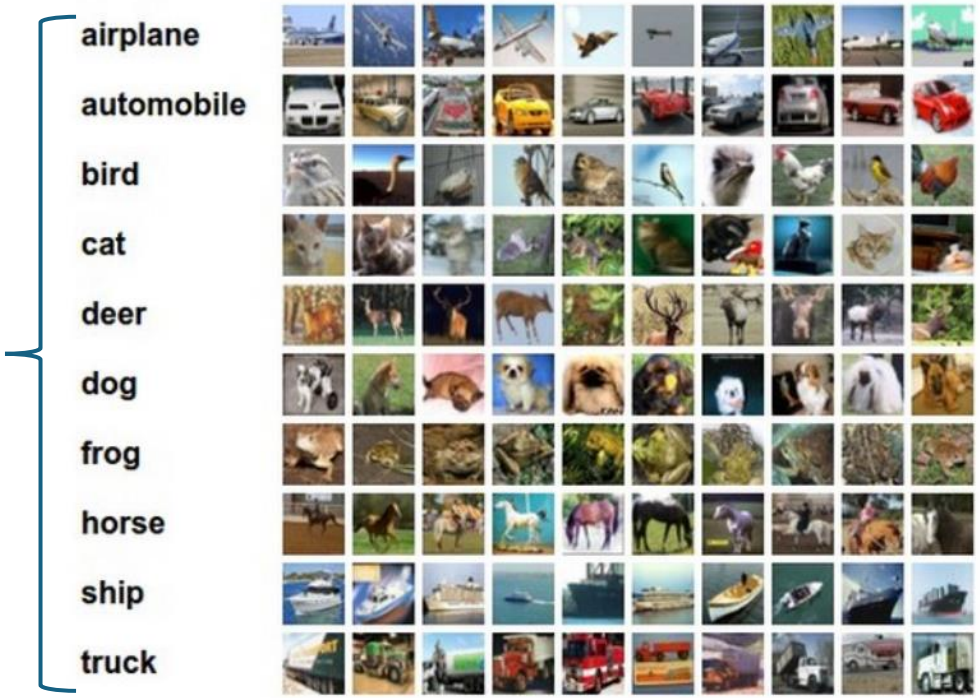
46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

→ 456

Image Classification with kNN

training examples CIFAR-10 data set

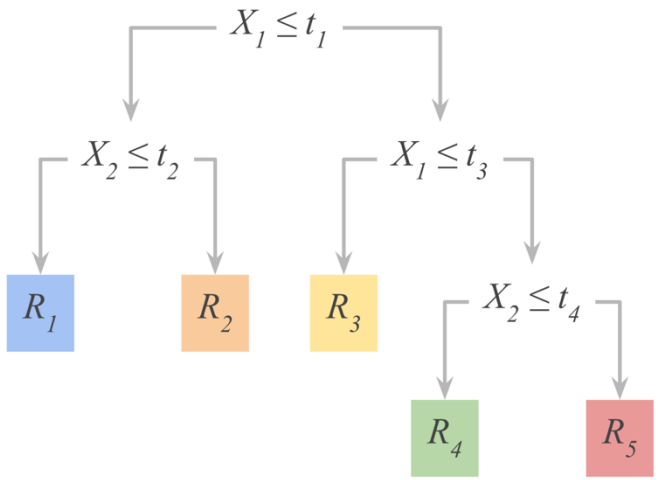
10 classes

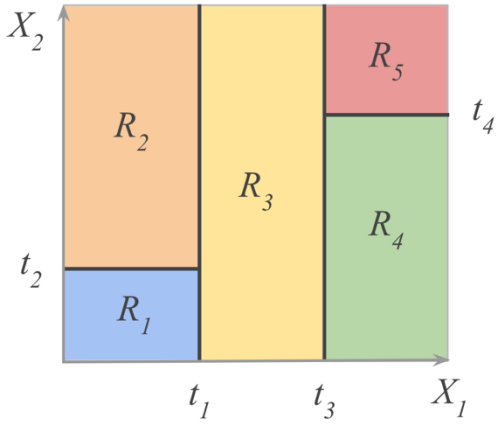


10 nearest neighbors to some test images

better than random guessing, but also not very impressive

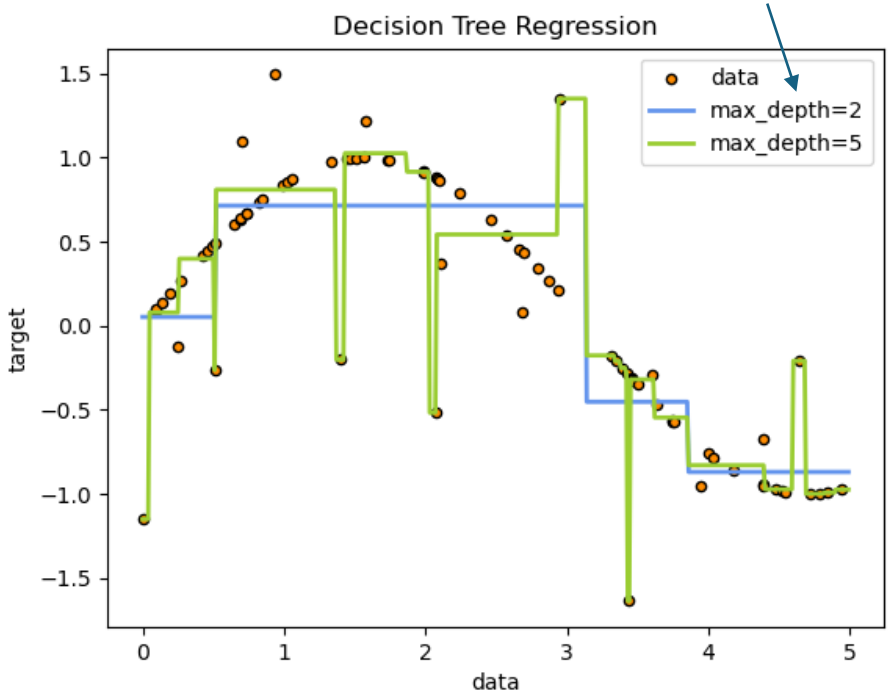
Decision Trees





- non-parametric learning of simple decision rules
- usually, binary trees
- axis-parallel decision boundaries (box-shaped regions in feature space)
- fit constant in each box
  - classification: majority class of target
  - regression: average of target
- fully explainable models

hyperparameter



scikit-learn

Ensemble Learning

idea: combine several individual models (often of the same type) to form an ensemble model with better predictive performance than each of its constituents

→ learning in several different ways from the training data

- the trainings of the individual constituent models are (kind of) independent
- outputs of individual models are combined (e.g., averaged)

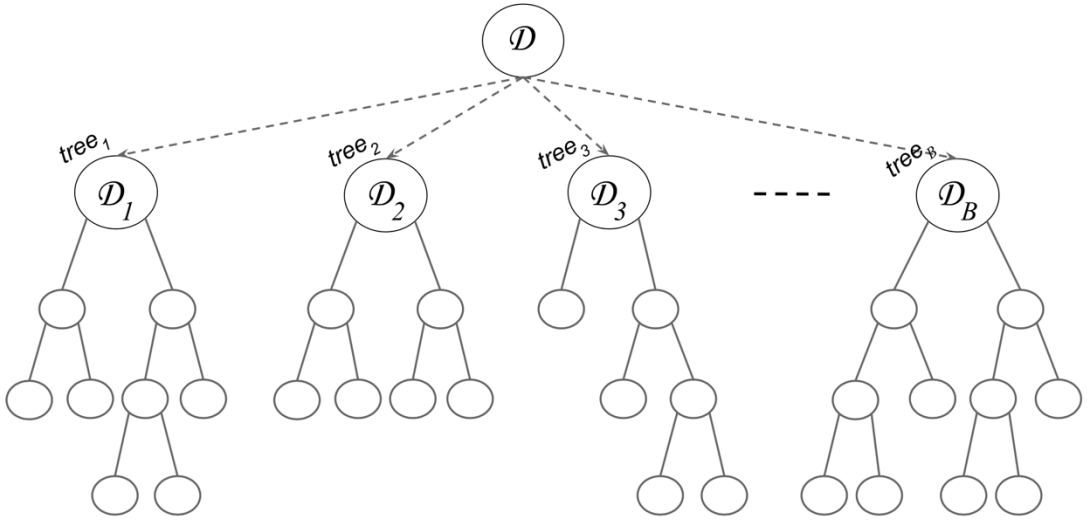
Bagging (Bootstrap Aggregating)

idea: train individual models of ensemble on random subsets of the training data (random sample with replacement)

- reduces variance of ensemble model compared to individual models
- but not bias → use low-bias base models

individual models combined by averaging (regression) or majority voting (classification) → committee of models

example with decision trees:



source

Random Subspace Method

aka feature bagging

idea: train different members (models) of an ensemble method on random subsets of all features (random sample with replacement)

→ reduce correlation between ensemble members

individual models combined by averaging (regression) or majority voting (classification) → committee of models

Random Forests

ensemble method using

- decision trees as individual models (usually, rather large, low-bias decision trees)
- combination of bagging and random subspace method (features sampled at each node in the trees)

compared to individual decision trees, random forests

+ reduce variance → improve accuracy

- lose explainability

one of the most popular off-the-shelf ML algorithms (often, good performance with little hyperparameter tuning and data preparation)

Boosting

idea: sequentially learn and combine several “weak” learners (such as small decision trees, but in principle any ML algorithm) to construct a “strong” one

→ gradually (in a greedy fashion) reducing bias of ensemble model

in simple terms:

building a model from the training data, then creating a second model that attempts to correct the errors from the first model, …

→ each subsequent weak learner is forced to concentrate on the examples that are missed by the previous ones in the sequence

not a committee of models:

typically, use simple, high-bias methods as individual models (e.g., small decision trees)

most prominent: AdaBoost, Gradient Boosting

Forward Stagewise Additive Modeling

boosting can be understood as fitting an additive expansion in a set of basis functions (e.g., decision trees): $f(\boldsymbol{x}) = \sum_{m=1}^{M} \gamma_m h_m(\boldsymbol{x})$

loss minimization of $f(\boldsymbol{x})$ computationally expensive → sequentially add and fit individual basis functions without changing already fitted ones

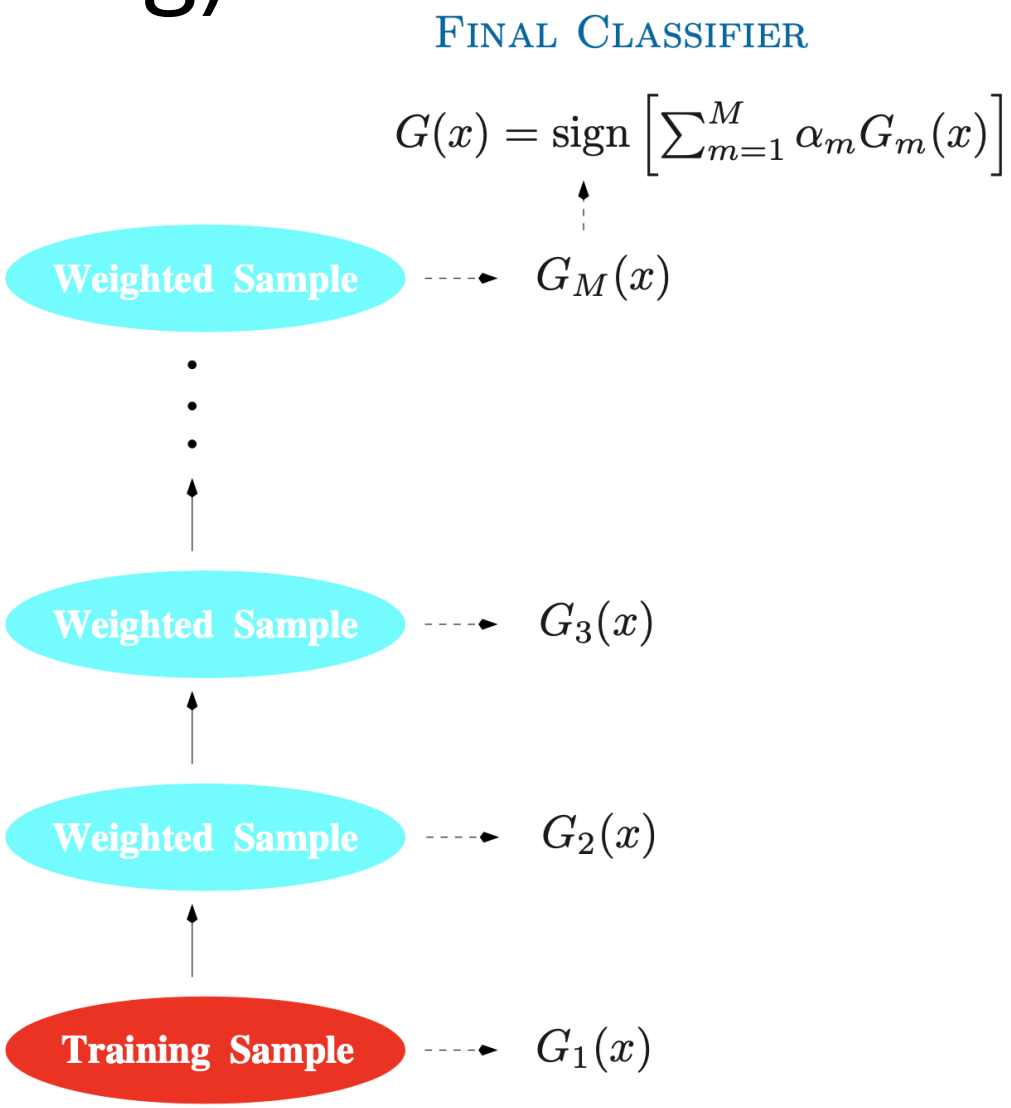
boosting algorithm:

for $m = 1$ to M:

1. compute $\underset{h_m, \gamma_m}{\operatorname{argmin}} \sum_{i=1}^{n} L\big(y_i, f_{m-1}(x_i) + \gamma_m h_m(x_i)\big)$ (initialized with $f_0(\boldsymbol{x}) = 0$)
2. set $f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \gamma_m h_m(\boldsymbol{x})$

AdaBoost (Adaptive Boosting)

- iteratively train weak classifiers on weighted data
- emphasizing misclassified points (weights increased for incorrect predictions, decreased for correct ones)
- combine them into weighted vote (forward stagewise additive modeling)



Gradient Boosting

cost function

loss function

define $\mathcal{L}(f_m) = \sum_{i=1}^{n} L\big(y_i, f_{m-1}(x_i) + \gamma_m h_m(x_i)\big)$

in forward stagewise additive modeling:

$$f_m(\boldsymbol{x}) = \sum_{m=1}^{M} \gamma_m h_m(\boldsymbol{x})$$

$$\underset{h_m, \gamma_m}{\operatorname{argmin}} \sum_{i=1}^{n} L\big(y_i, f_{m-1}(x_i) + \gamma_m h_m(x_i)\big)$$ for each stage

→ can be hard to solve for general loss function L

idea: approximation of $\mathcal{L}$ by its Taylor expansion

$$\mathcal{L}(f_m) \approx \mathcal{L}(f_{m-1}) + \sum_{i=1}^{n} h_m(x_i) \cdot \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

gradient g at x_i:
constant for this iteration

→ decreased by going a step of size γ_m (potentially optimized via line search) in the direction $h_m(\boldsymbol{x}) \approx -g$ (functional gradient descent)

→ find h_m by fitting a regression model with targets $-g$ (typically decision tree regressors of fixed size: gradient-boosted decision trees)

works like this for both regression and classification tasks (just different loss functions)

Decision Tree Sizes for Boosting

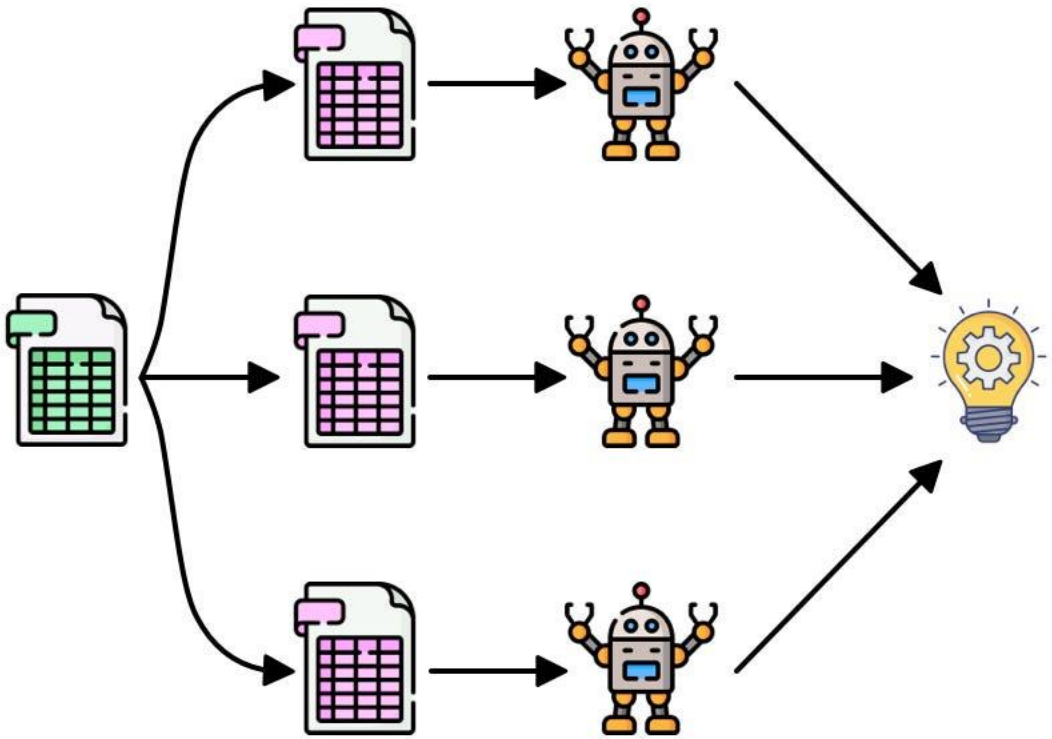
consider degree to which features interact with each other in given data set

- decision trees with single split (aka decision stumps): covering no interaction effects, just main effects of individual features
- decision trees with two splits: covering second-order interactions
- decision trees with three splits: covering third-order interactions

(usually, interaction order not much higher than ~5 in real-world data sets)

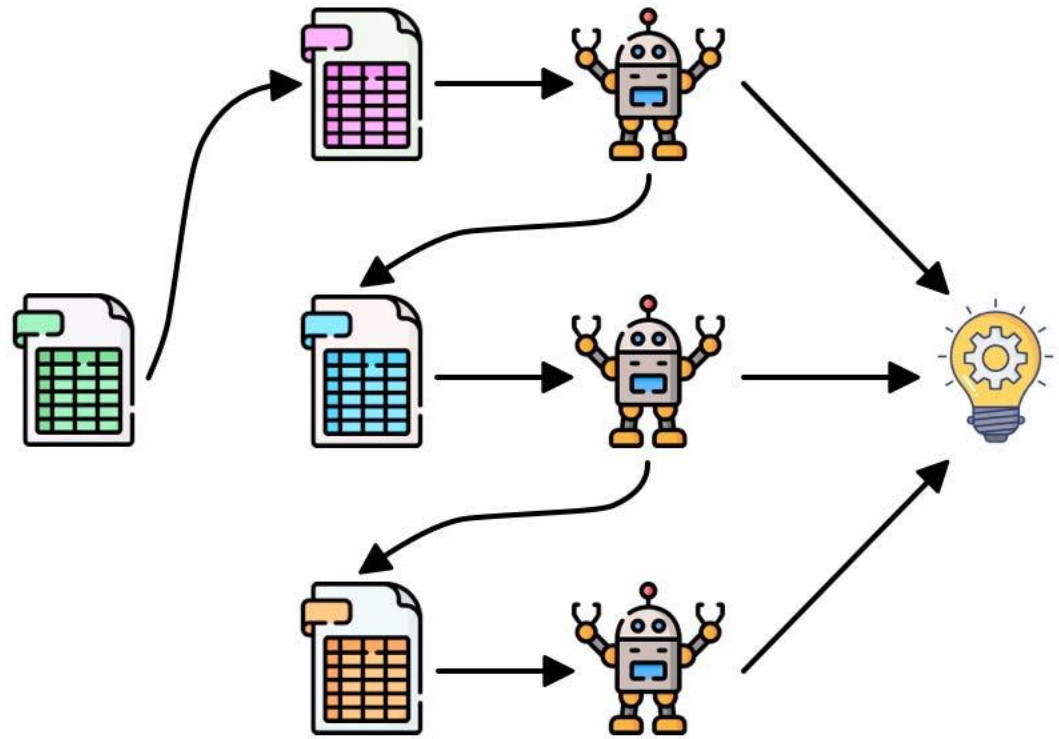
why boosting often works better than single large, low-bias model:
uncorrelated learners, each focusing on a specific aspect of the data

Bagging

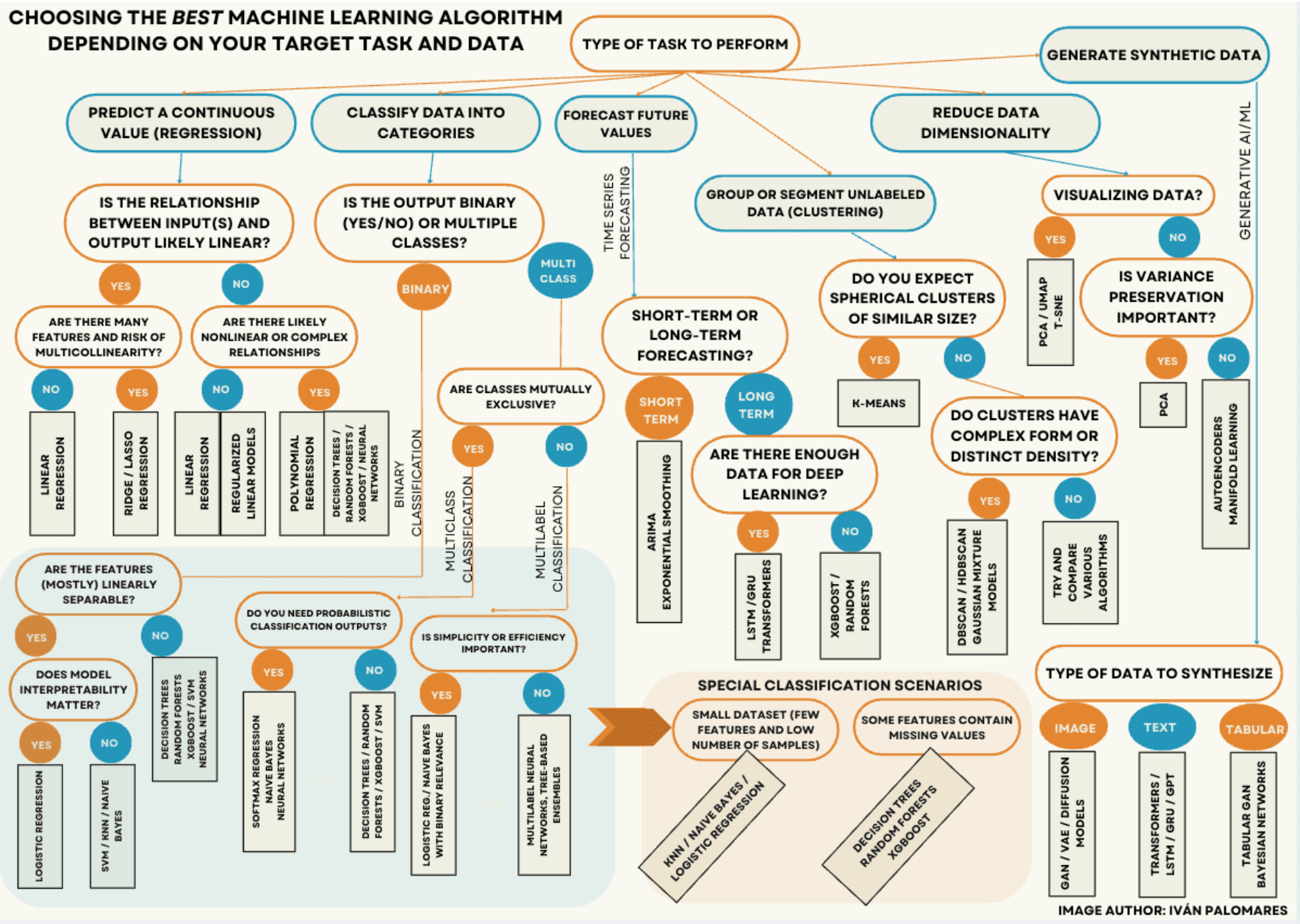


Parallel

Boosting



Sequential



no universally best model

choice depends on

- data characteristics
- problem requirements
- interpretability needs
- computational constraints

rule of thumb:

- for unstructured data (text, images/video, audio) → deep learning
- for tabular data → tree-based ensembles (gradient boosting, random forests)